LIMKOKWING UNIVERSITY OF CREATIVE TECHNOLOGY
SIERRA LEONE

FACULTY OF INFORMATION AND COMMUNICATION TECHNOLOGY

DATA STUCTURE & AGORITHM ANALYSIS (DCOMP320)

Title : Stacks Implementation
Issue Date : Week 2
Due Date : Week 4
Lecturer/Examiner : Miss. Hawa Kallon
Name of Student : Zainab Mariatu Sesay
Student ID No. : 905005586
Class : BSEM2102
Semester/Year : 5/3

Academic Honesty Policy Statement

I/We, hereby attest those contents of this attachment are my own work. Referenced works, articles, art, programs, papers or parts thereof are acknowledged at the end of this paper. This includes data excerpted from CD-ROMs, the Internet, other private networks, and other people's disk of the computer system.

Student's Signature: Date: ,October, 2026

LECTURER'S COMMMENTS/GRADE:	for office use only upon receive Remark DATE : TIME : RECEIVER'S NAME :

TABLE OF CONTENTS

INTRODUCTION 3
DATA STRUCTURES USED 4
PROGRAM DESIGN 5
TESTING AND RESULTS 9
CONCLUSION 11
REFERENCES 12

INTRODUCTION

Stacks are one of the most fundamental data structures in computer science. They operate on the principle of Last-In-First-Out (LIFO)**,** meaning the last element pushed onto the stack is the first one to be popped out. This property makes stacks particularly useful in parsing and evaluating mathematical expressions, handling function calls, and managing undo operations in applications.

In this assignment, a Python program was developed to evaluate mathematical expressions using a stack-based approach**.** The program reads expressions from an input file, applies the Shunting Yard algorithm to handle operator precedence and parentheses, and writes the results to an output file. This demonstrates the practical importance of stacks in computation.

DATA STRUCTURES USED

The primary data structure used in this program is the **Stack**.

- **Stack Class Implementation**
    - push(item) → Adds an element to the stack.
    - pop() → Removes and returns the top element.
    - peek() → Returns the top element without removing it.
    - is_empty() → Checks if the stack is empty.

Stacks are used in two key ways:

1. **Operand Stack** → Stores numbers during evaluation.
2. **Operator Stack** → Stores operators and parentheses, ensuring correct precedence and order of operations.

This dual-stack approach ensures that expressions are evaluated correctly, respecting mathematical rules.

PROGRAM DESIGN

The program is divided into three main components:

Stack Class: Implements basic stack operations.

- Implements the fundamental stack operations: push, pop, peek, and is_empty.
- Internally uses a Python list to store elements.
- Provides the foundation for managing both operators and operands during expression evaluation.

Expression Evaluation Function: Uses the Shunting Yard algorithm to parse and evaluate expressions.

- Handles operators (+, -, *, /).
- Manages parentheses for grouping.
- Applies operator precedence rules.

File Handling Functions

- Reads expressions from input.txt.
- Evaluates each expression.
- Writes results to output.txt, preserving separators (-----).

Pseudocode

Class Stack:

Initialize empty list

Define push, pop, peek, is_empty methods

Function evaluate_expression(expr):

Initialize values stack and ops stack

For each character/token in expr:

If number: push to values stack

If operator or parenthesis: manage precedence and apply operators using stacks

Return final computed value from values stack

Function process_files():

Open input.txt for reading and output.txt for writing

For each line in input.txt:

If line is separator or empty: write directly to output.txt

Else: evaluate expression and write result to output.tx

Stack Implemntation code

```
class Stack:
    def __init__(self):
        self.stack = []

    def push(self, item):
        self.stack.append(item)

    def pop(self):
        if not self.is_empty():
            return self.stack.pop()
        return None

    def peek(self):
        if not self.is_empty():
            return self.stack[-1]
```

```
        return None

    def is_empty(self):
        return len(self.stack) == 0


def precedence(operator_symbol):
    if operator_symbol in ('+', '-'):
        return 1
    if operator_symbol in ('*', '/'):
        return 2
    return 0

def apply_operator(a, b, operator_symbol):
    if operator_symbol == '+': return a + b
    if operator_symbol == '-': return a - b
    if operator_symbol == '*': return a * b
    if operator_symbol == '/': return a / b

def evaluate(expression_text):
    value_stack = Stack()
    operator_stack = Stack()
    i = 0 #character_position

    while i < len(expression_text):
        if expression_text[i] == ' ':
            i += 1
            continue

        elif expression_text[i].isdigit():
            number_value = 0
            while i < len(expression_text) and expression_text[i].isdigit():
                number_value = number_value * 10 + int(expression_text[i])
                i += 1
            value_stack.push(number_value)
            continue

        elif expression_text[i] == '(':
            operator_stack.push(expression_text[i])

        elif expression_text[i] == ')':
            while not operator_stack.is_empty() and operator_stack.peek() != '(':
                number_value1 = value_stack.pop()
                number_value2 = value_stack.pop()
                pending_operator = operator_stack.pop()
```

```
                value_stack.push(apply_operator(number_value2, number_value1,
pending_operator))
            operator_stack.pop()

        else:
            while (not operator_stack.is_empty() and
precedence(operator_stack.peek()) >= precedence(expression_text[i])):
                number_value1 = value_stack.pop()
                number_value2 = value_stack.pop()
                pending_operator = operator_stack.pop()
                value_stack.push(apply_operator(number_value2, number_value1,
pending_operator))
            operator_stack.push(expression_text[i])
        i += 1

    while not operator_stack.is_empty():
        number_value1 = value_stack.pop()
        number_value2 = value_stack.pop()
        pending_operator = operator_stack.pop()
        value_stack.push(apply_operator(number_value2, number_value1,
pending_operator))

    return value_stack.pop()

def process_files():
    with open("Input.txt", "r") as input_file, open("Output.txt", "w") as
output_file:
        for input_line in input_file:
            expression_line = input_line.strip()
            if expression_line == "-----":
                output_file.write("-----\n")

            elif expression_line:
                try:
                    calculated_result = evaluate(expression_line)
                    output_file.write(str(calculated_result) + "\n")
                except Exception:
                    output_file.write("Error in expression\n")

if __name__ == "__main__":
    process_files()
```

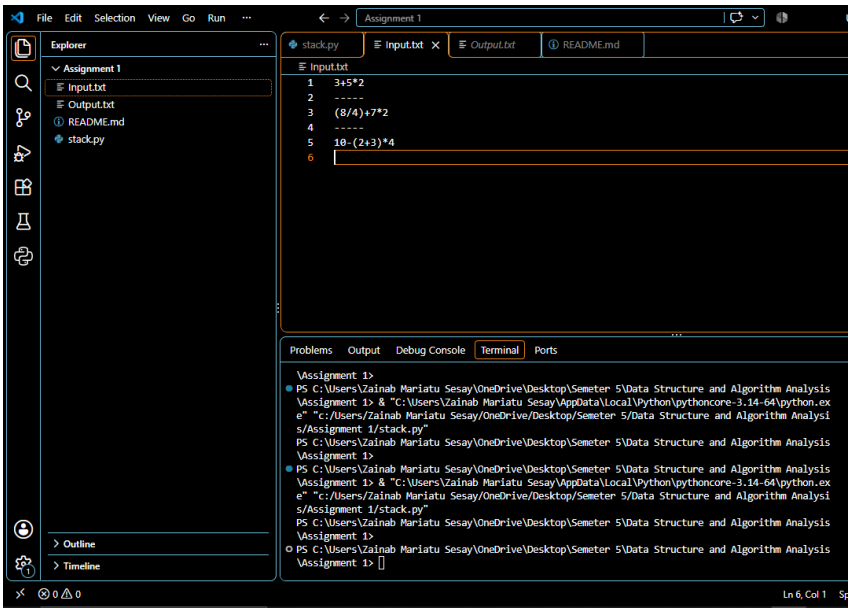
TESTING AND RESULTS

The program was tested using a sample input.txt file containing multiple mathematical expressions. The goal was to verify that:

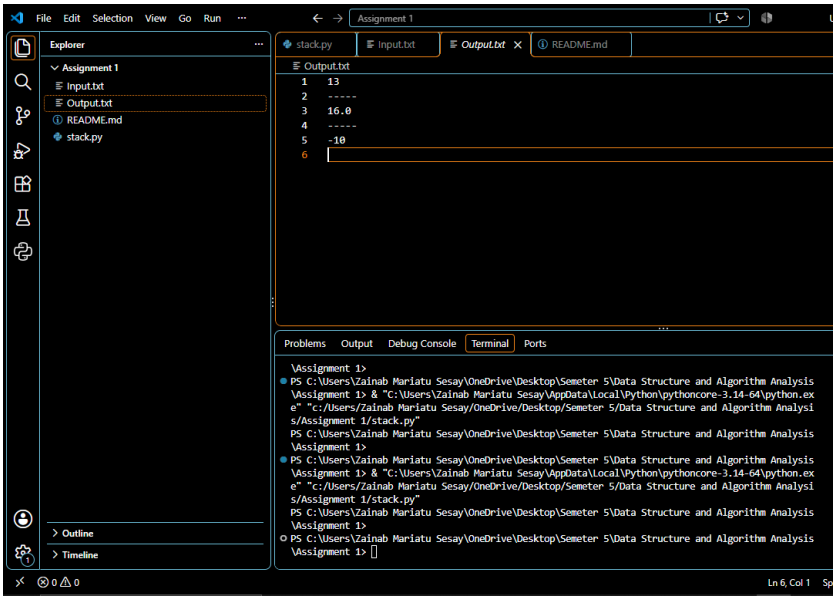
- Operators follow correct precedence (* and / before + and -).
- Parentheses are handled properly.
- Separator lines (-----) are preserved in the output.
- Invalid expressions are caught and reported as "Error in expression".

Testing was performed by running the program with different sets of expressions and comparing the generated output.txt with expected results

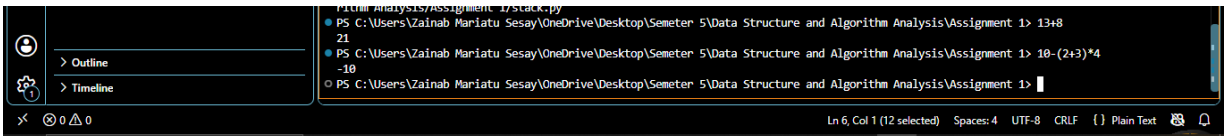
Sample Input (Input.txt):



Sample Output (Output.txt):



The terminal was also used to test and it provided similar result.



The results confirm that the program correctly handles:

- Basic arithmetic operations.
- Parentheses.
- Operator precedence.
- Separators between expressions.

Invalid expressions are caught and return

CONCLUSION

This assignment demonstrates the importance of stacks in evaluating mathematical expressions. By implementing the Shunting Yard algorithm, the program ensures correct handling of operator precedence and parentheses. The use of stacks makes the solution efficient, structured, and easy to extend.

Stacks are widely used in compilers, interpreters, and calculators, and this project provides a practical example of their application in real-world computing tasks.

REFERENCES

Contributors, W., 2026 . Shunting yard algorithm. [Online]
Available at: https://en.wikipedia.org/wiki/Shunting_yard_algorithm
[Accessed 12 October 2026].

Mcleod, J., 2023. Implementing the Python Shunting Yard Algorithm. [Online]
Available at: https://www.martinbroadhurst.com/shunting-yard-algorithm-in-python#Python_Implementation
[Accessed 13 October 2026].

Oparin, E. S., 2025. Shunting-Yard Algorithm: A Simple Approach to Parsing Infix Expressions. [Online]
Available at: https://every-algorithm.github.io/2025/01/21/shunting-yard_algorithm.html
[Accessed 14 October 2026].